

Problème des Lecteurs / Rédacteurs

Synchronisation des processus et des threads

1. Énoncé du problème

Plusieurs processus concurrents accèdent à une ressource partagée (fichier, base de données...). Deux types d'accès sont possibles :

#	Règle	Implication
1	Plusieurs lecteurs peuvent lire en même temps	Accès concurrent en lecture autorisé (degré ≥ 1)
2	Un seul rédacteur à la fois	Exclusion mutuelle totale en écriture
3	Lecteurs prioritaires sur les rédacteurs	Un rédacteur ne peut pas passer devant un lecteur actif
4	Le rédacteur attend le DERNIER lecteur	$nb_lect == 0$ avant d'entrer en section critique

2. Variables de synchronisation

Variable	Type	Init	Rôle
nb_lect	entier	0	Compte les lecteurs actifs
mutex	sémaphore binaire	1	Protège nb_lect (section critique du compteur)
ecriture	sémaphore binaire	1	Bloque l'accès en écriture (exclusion totale)

3. Algorithme du LECTEUR

ENTRÉE : $P(mutex) \rightarrow nb_lect++ \rightarrow \text{si } nb_lect == 1 : P(ecriture) \rightarrow V(mutex)$

SECTION CRITIQUE : Lecture libre — plusieurs lecteurs simultanés autorisés

SORTIE : $P(mutex) \rightarrow nb_lect-- \rightarrow \text{si } nb_lect == 0 : V(ecriture) \rightarrow V(mutex)$

Explication pas à pas :

1er lecteur ($nb_lect == 1$) Exécute $P(ecriture) \rightarrow$ bloque tout rédacteur. Les lecteurs suivants passent directement.

Lecture simultanée Plusieurs lecteurs lisent en même temps. Aucun rédacteur ne peut entrer.

**Dernier lecteur
(nb_lect==0)**

Exécute $V(\text{écriture}) \rightarrow$ libère les rédacteurs en attente.

Code C — Fonction lecteur :

[illegible]

4. Algorithme du RÉDACTEUR

ENTRÉE : P(ecriture) → attend que nb_lect == 0

SECTION CRITIQUE : Écriture exclusive — aucun autre thread n'a accès

SORTIE : V(ecriture) → libère la ressource

ENTRÉE : P(ecriture) → attend que nb_lect == 0

SECTION CRITIQUE : Écriture exclusive — aucun autre thread n'a accès

SORTIE : V(ecriture) → libère la ressource

ENTRÉE : P(ecriture) → attend que nb_lect == 0

SECTION CRITIQUE : Écriture exclusive — aucun autre thread n'a accès

SORTIE : V(ecriture) → libère la ressource

Explication :

Le rédacteur appelle simplement **P(ecriture)**. Ce sémaphore est maintenu à 0 tant qu'au moins un lecteur est actif (le 1er lecteur l'a pris). Le rédacteur se bloque donc automatiquement jusqu'à ce que le **dernier lecteur** exécute **V(ecriture)**. Une fois entré, il est seul — aucun lecteur ni autre rédacteur ne peut interférer.

Code C — Fonction rédacteur :

```
void *redacteur(void *arg) {  
    int id = *(int *)arg;  
    for (int i = 0; i < NB_ITERATIONS; i++) {  
        sleep(2);  
  
        /* ■■ ENTREE ████████████████████████████████████████ */  
        sem_wait(&ecriture); // attend nb_lect == 0  
  
        /* ■■ SECTION CRITIQUE : ECRITURE ████████████████ */  
        donnee_partagee++;  
        printf("[Redacteur %d] Ecrit: donnee = %d\n", id, donnee_partagee);  
        sleep(1);  
  
        /* ■■ SORTIE ████████████████████████████████████████ */  
        sem_post(&ecriture);  
    }  
    return NULL;  
}
```

5. Fonction main — Initialisation et lancement

```

int main(void) {
// Initialisation des semaphores
sem_init(&mutex, 0, 1); // mutex binaire
sem_init(&ecriture, 0, 1); // acces exclusif ecriture

pthread_t tl[NB_LECTEURS], tr[NB_REDACTEURS];
int ids_l[NB_LECTEURS], ids_r[NB_REDACTEURS];

// Creation des threads lecteurs
for (int i = 0; i < NB_LECTEURS; i++) {
ids_l[i] = i + 1;
pthread_create(&tl[i], NULL, lecteur, &ids_l[i]);
}
// Creation des threads redacteurs
for (int i = 0; i < NB_REDACTEURS; i++) {
ids_r[i] = i + 1;
pthread_create(&tr[i], NULL, redacteur, &ids_r[i]);
}
// Attendre la fin de tous les threads
for (int i = 0; i < NB_LECTEURS; i++) pthread_join(tl[i], NULL);
for (int i = 0; i < NB_REDACTEURS; i++) pthread_join(tr[i], NULL);

sem_destroy(&mutex);
sem_destroy(&ecriture);
return 0;
}

```

6. Trace d'exécution commentée

Sortie console	Explication
[Lecteur 2] Lit : donnee=0 actifs=1	1er lecteur → P(ecriture) pris
[Lecteur 3] Lit : donnee=0 actifs=2	2e lecteur → entre directement
[Lecteur 1] Lit : donnee=0 actifs=3	3e lecteur → lecture simultanée ■
[Lecteur 4] Lit : donnee=0 actifs=4	4e lecteur → lecture simultanée ■
[Lecteur 5] Lit : donnee=0 actifs=5	5e lecteur → lecture simultanée ■
[Redacteur 2] Ecrit: donnee = 1	nb_lect=0 → rédacteur entre, seul ■
[Redacteur 1] Ecrit: donnee = 2	rédacteur suivant, exclusif ■
=== Fin. Valeur finale=6 Attendu=6 ===	Cohérence parfaite ■

7. Résumé — Points clés

Point clé	Mécanisme utilisé	Garanti ?
-----------	-------------------	-----------

Lectures simultanées	$\text{nb_lect} > 0 \rightarrow$ plusieurs threads en SC lecture	■
Exclusion en écriture	P(ecriture) avant SC écriture	■
Priorité aux lecteurs	1er lecteur prend ecriture $\rightarrow$ bloque rédacteurs	■
Rédacteur attend dernier	$\text{nb_lect} == 0 \rightarrow$ V(ecriture) par dernier lecteur	■
Protection de nb_lect	Toute modification de nb_lect sous mutex	■

```

# Compilation
gcc -o lect_red lecteurs_redacteurs.c -lpthread

# Execution
./lect_red

```